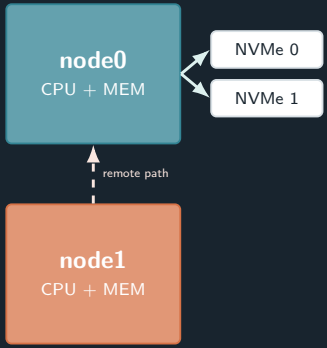
SCHED_EXT · LOCK-AWARE SCHEDULING · NUMA/I/O CO-DESIGN

LockCoop 端到端性能双稳态的体系结构归因

双路 ARM NUMA 服务器上的调度、内存放置与 NVMe/O_DIRECT 路径



1 RESEARCH NARRATIVE

问题与实验系统

先给结论：这不是随机崩溃，而是两个可重复的物理状态

~ 5.0–5.8k TPS
有利联合局部态：DB CPU/锁/内存靠近 node0 I/O

~ 1.5–1.6k TPS
不利局部态：工作集或锁热点落入 node1 路径

$r = 0.881$
20 runs：TPS 与 node0 anchor 比例相关

自然观察
20 个未绑定 `scx_lockcoop` runs 出现双峰

→ **相关证据**
anchor node 与 TPS 同步

→ **干预证据**
DB node / CPU×MEM 反事实

已证实 DB 的 NUMA 放置决定性能档位。

强证据 node0 内存显著改善当前 O_DIRECT 路径。

待细分 PCIe、IRQ、IOMMU/SMMU、固件各层贡献。

结论 LockCoop 会强力保持首次形成的状态，无论其空间局部性是否有利。

研究问题：微基准有效，为什么 OLTP 端到端结果会分叉？

原始预期

锁竞争微基准表明 lockcoop placement 能缩短 holder/waiter 交接路径，因此 MariaDB 高并发 OLTP 应获得稳定提升。

观测冲突

相同二进制、参数、数据集和线程数下，单次 run 可处于 5k TPS，也可停留在 1.5k TPS；算术均值和 CV 难以解释其机制。

Q1 · 统计性质 双峰是异常值、时间漂移，还是稳定状态？

Q2 · 模式开关 client、DB CPU、锁 anchor、内存、存储谁决定档位？

Q3 · 算法反馈 `scx_lockcoop` 如何维持初始 NUMA 状态？

Q4 · 体系结构边界 能定位到哪一级，哪些仍需 PMU/IRQ/IOMMU 实验？

结论 研究目标不是寻找一个“好看的平均数”，而是建立可干预、可证伪的跨层因果模型。

排查路线：从版本对齐到空间反事实



方法论升级： observation → correlation → intervention → subsystem isolation。
每一步都改变一个受控因素，并记录吞吐、p95、NUMA 内存、InnoDB physical reads 与调度器 anchor。

实验平台：双路 256 核 ARM NUMA，而不是单节点服务器

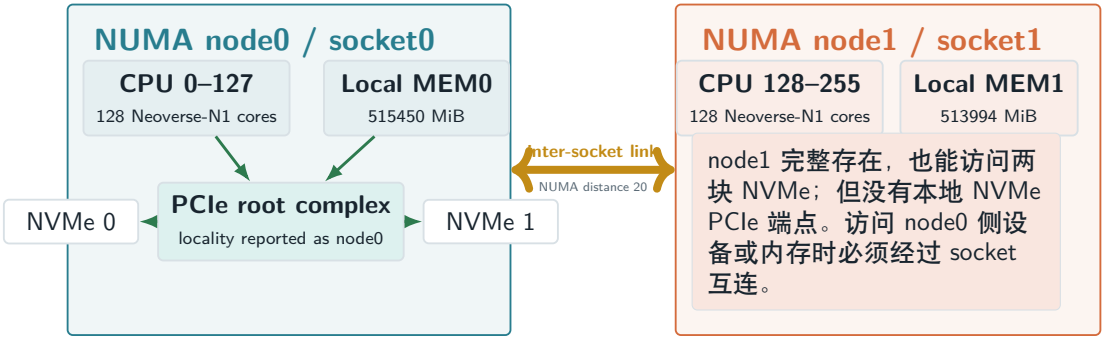
项目	配置
CPU	2 × Ampere Altra Max M128-30, Neoverse-N1, 3.0 GHz
核心	256 cores / 256 CPUs, 1 thread/core
NUMA	node0: CPU 0–127; node1: CPU 128–255
内存	node0 515450 MiB; node1 513994 MiB
距离	local 10; remote 20
存储	2 × Intel SSDPF2KX960HZ 894.3 GiB NVMe

项目	配置
OS	openEuler 24.03 LTS, aarch64
Kernel	6.17.0-lockcoop -scx617, sched_ext
Database	MariaDB 10.6.27, 128 MiB buffer pool
I/O mode	`O_DIRECT`
Dataset	16 tables × 1M rows, 约 4.5 GiB
Client	sysbench 1.0.20, OLTP read/write
LC args	`-A -m 10 -t 2 -M`

结论 buffer pool 仅为数据集的约 2.8%，持续物理读使 CPU/内存/设备的空间关系进入端到端关键路径。

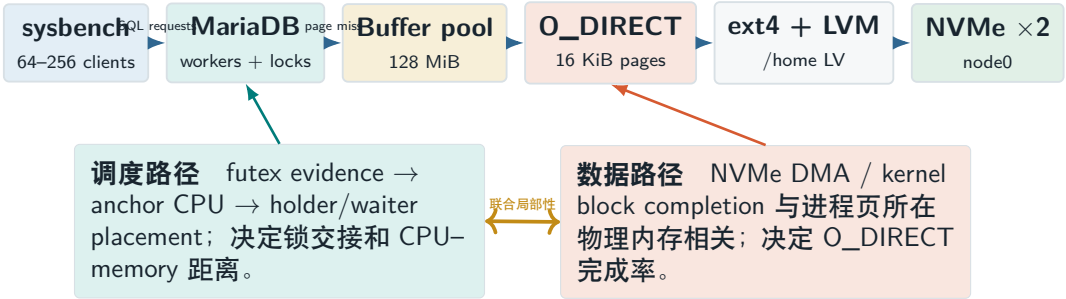
数据来源：`lscpu`、`numactl --hardware`、`lsblk`、MariaDB variables；2026-08-02 现场快照。

物理拓扑：NUMA node 与 NVMe 设备近邻



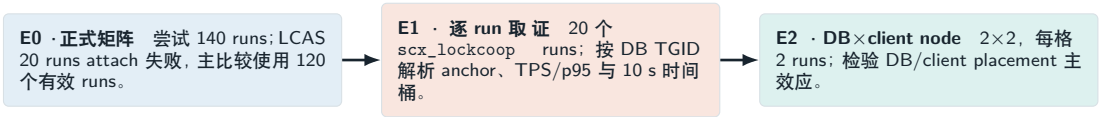
数据来源：sysfs: `nvme{0,1}/device/numa_node=0`；两块设备共同承载 `/home` LVM。

端到端数据路径：调度局部性与 I/O 数据移动在 MariaDB 内汇合



结论 只观察“线程在哪个 CPU”不够；必须同时记录锁 anchor、进程内存页和设备拓扑。

实验设计：自然样本、受控干预与子系统隔离



结论 证据顺序是自然观察 → 相关性 → 因果干预 → 子系统隔离。

共同协议、观测量与有效性判定

控制变量

MariaDB 10.6.27、相同 datadir、128 MiB buffer pool、O_DIRECT、诊断阶段 128 threads、`scx_lockcoop -A -m10 -t2 -M`。

状态判定

不删除 5k 或 1.5k 样本；二者对应不同物理状态。2k–3k 样本回看 10 s 时间序列，判断是否迁移。

观测量

TPS、p95、InnoDB physical reads/s、reads/transaction、DB NUMA pages、实验 TGID 的 anchor-node0 比例。

SCX 有效性

必须同时满足 scheduler PID 存活、kernel state=enabled、attach signature 存在。LCAS 未满足，明确标为无效。

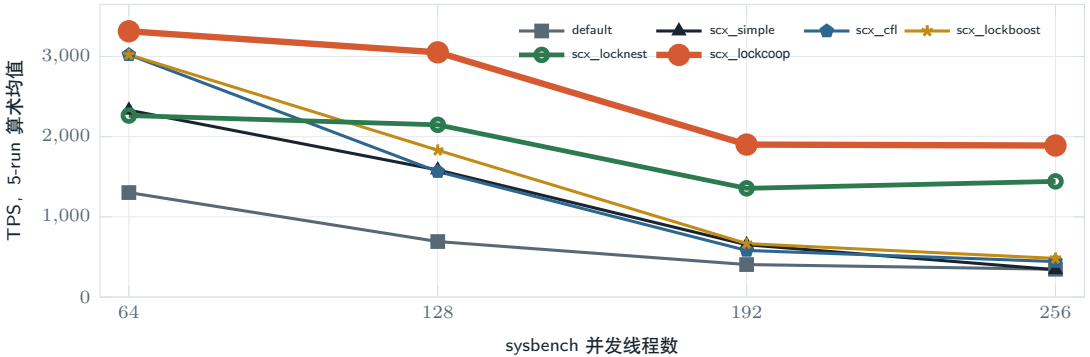
结论 不能以 sysbench 输出文件存在判定成功；还必须确认调度器实际接管内核。

2 RESEARCH NARRATIVE

端到端现象与双稳态证据

有效正式矩阵：LockCoop 均值领先，但均值掩盖了状态混合

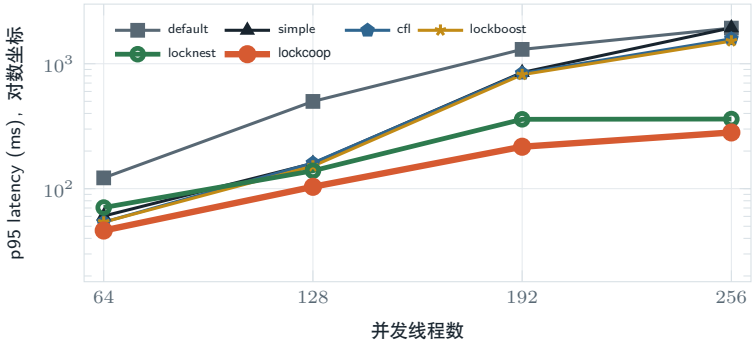
端到端现象与双稳态证据



结论 256 threads 下 scx_lockcoop 均值为 1889.80 TPS，是 default 的 5.44×、scx_simple 的 5.54×；但 5-run mean 不是单一稳态的代表值。

数据来源：正式 tag paper53_unbound_no_nest_runs5_20260801_224447；图中 6 个有效系列共 120 runs。LCAS 的 20 个 fallback 样本不绘制。

尾延迟：LockCoop 在高并发下仍保持明显优势



281.76 ms
LockCoop @ 256T

1928.11 ms
default @ 256T

1938.12 ms
scx_simple @ 256T

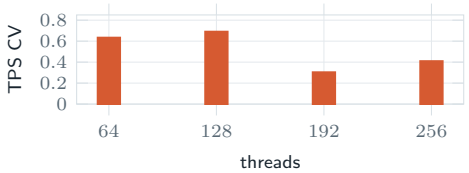
结论 吞吐提升与 p95 降低方向一致；但这里的 p95 也是 5 次 run 的均值，仍需按物理状态分层解释。

为什么 CV 很大：它度量“混合状态”，不是普通噪声

CV 定义

$$\mathrm{CV} = \frac{s_{\mathrm{TPS}}}{\overline{\mathrm{TPS}}}$$

例如 128-thread `scx_lockcoop` ：$\bar{x} = 3051.76$，$s = 2088.99$，所以 $\mathrm{CV} = 0.685$。



高态样本 4.9k–5.8k TPS；anchor 100% node0。

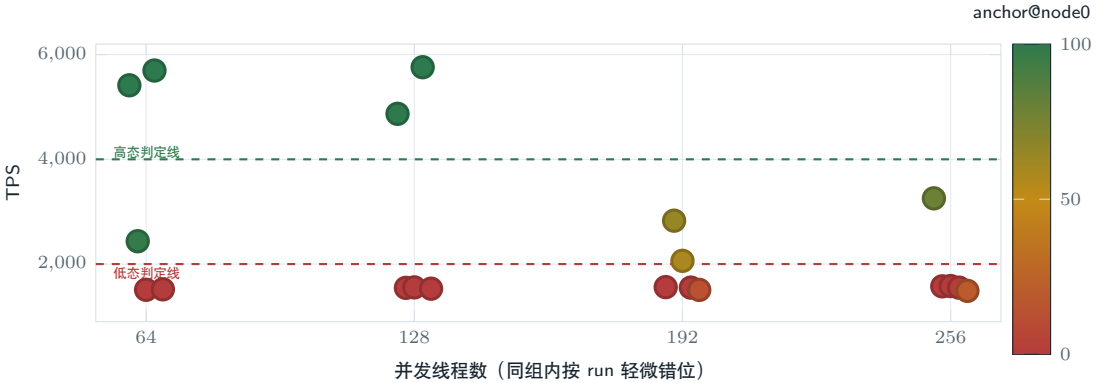
低态样本 1.49k–1.58k TPS；anchor 几乎全部 node1。

迁移样本 60 s 窗口内从高态向低态移动；平均落在 2k–3k。

统计含义 单个高 CV 无法区分噪声与双峰；必须回到逐 run 与时间序列。

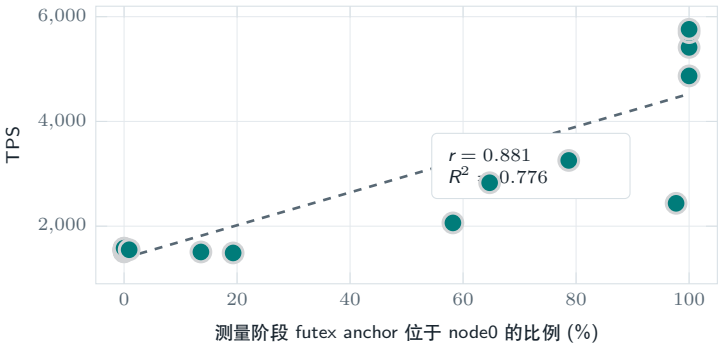
结论 不能用 CV 阈值删除某一档；应把 NUMA/anchor 状态作为实验因子分层。

20 个 LockCoop runs：每个点都是一个完整 60 秒实验



结论 4 个 >4k TPS 样本全部为 100% node0 anchor；12 个 <2k TPS 样本的 node0 anchor 平均仅 2.8%。

相关性证据：anchor 是联合 NUMA 状态的高质量代理变量



支持 anchor-node 与端到端吞吐强相关。

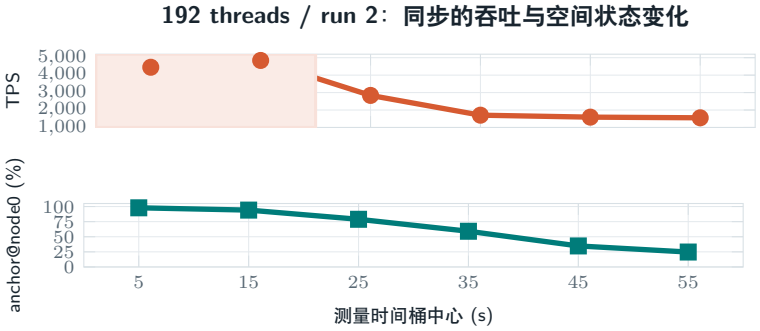
不充分 相关性不能证明 anchor 单独导致高态。

自然耦合 未绑定启动时 CPU、first-touch memory 与 anchor 通常同 node。

需要干预 必须拆开 CPU/MEM/anchor 才能判断充分条件。

结论 这张图用于提出机制假设；真正的因果判断来自后续受控 2×2 和 CPU×MEM 反事实。

时间证据：中间值是状态迁移，不是稳定的“第三档”



前 20 秒

4452 / 4842 TPS
anchor: 98% / 94%

后 20 秒

1592 / 1554 TPS
anchor: 35% / 25%

60 秒均值

2828.83 TPS
是轨迹积分，不是稳态中心。

从现象得到的可检验预测

H1 · client 主导 若正确，交换 sysbench node 应交换高低档。

✗ 2×2 否定

H2 · DB 放置主导 固定 DB node 后，client 不应改变档位。

✓ DB node 是自然开关

H3 · anchor 充分 anchor=node0 时，即使 MEM1 仍应高。

✗ C0/M1 仅 1.33k

H4 · 联合局部性 compute/lock/MEM/I/O 对齐才进入高态。

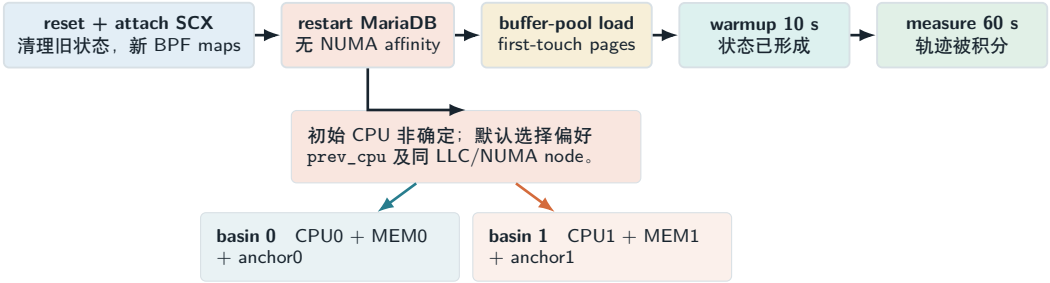
✓ 四象限支持

结论 下一部分使用干预实验区分“观测指标”和“充分因果条件”。

3 RESEARCH NARRATIVE

跨层因果诊断

初始条件从哪里来：runner 先 attach 调度器，再启动数据库



结论 64/128-thread 中 warmup TPS 与 measure TPS 的相关系数为 0.990/0.998：档位通常在测量前已经选定。

数据来源：`run_full_sweep.sh:454--490`；sched_ext CPU selection 路径。

LockCoop 机制（一）：首次证据创建 anchor，热锁不断续期


```
249 static void init_lock_nest(..., s32 holder_cpu, ...)
250 {
251     n_info->anchor_cpu = holder_cpu;
252     n_info->last_update_ts = now;
253     n_info->handoff_seq = 1;
254 }
255
256 /* Hot futex updates this timestamp every time. */
257 n_info->last_update_ts = now;
258 n_info->handoff_seq++;
```

数据来源：scx_lockcoop .bpf.c:249--262, 328--368

第一次 futex waker/holder 证据 直接把当前 CPU 写入 `anchor_cpu`。

↓

`-m 10` 阈值是 10 次状态更新，不是 10 次经验证的锁 handoff。

↓

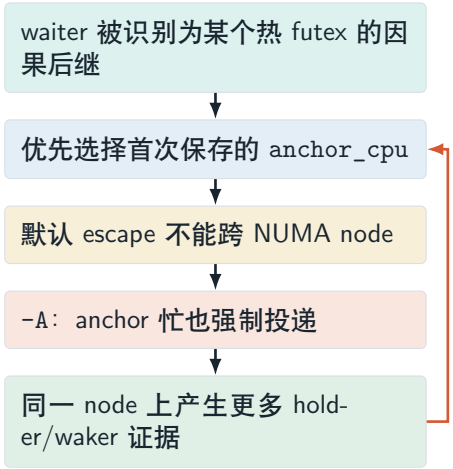
`-t 2` + **默认** `-e 2000` 前者把高竞争阈值设为 2 waiters；后者才是 2 秒 expiry。热锁持续刷新时间戳，几乎不会过期。

结论 繁忙 futex 的首次 anchor 具有很强的历史依赖，启动时的一次空间选择可长期存在。

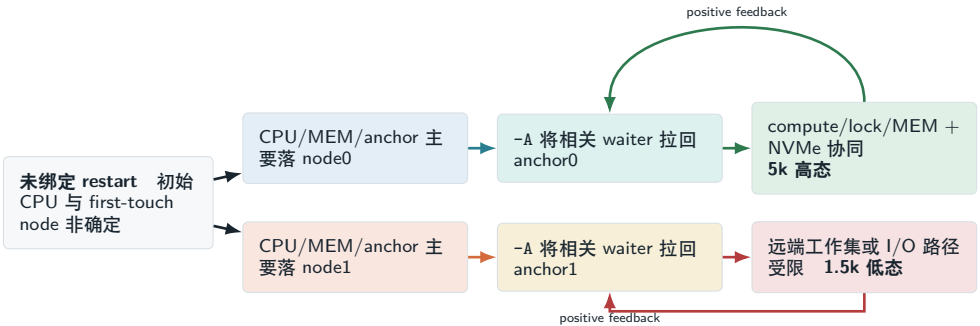
LockCoop 机制（二）：同 node escape 与 -A 构成正反馈


```
654 if (n_info->handoff_seq < min_handoff_seq)
655     goto fallback;
656
657 cpu = n_info->anchor_cpu;
658 if (anchor_is_idle(cpu))
659     dispatch_local(cpu);
660
661 if (same_topology_node(cpu, prev_cpu) &&
662     escape_budget_available(n_info))
663     try_prev_cpu(prev_cpu);
664
665 if (force_busy_anchor) /* -A */
666     dispatch_local(cpu); /* even when busy */
```

数据来源：scx_lockcoop .bpf.c:654--697



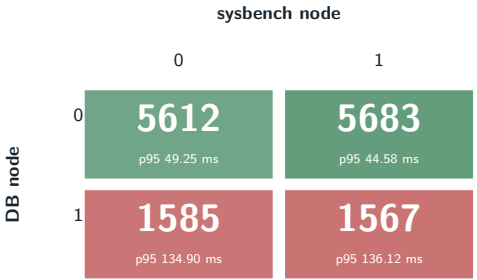
算法与体系结构耦合：LockCoop 放大“好状态”，也放大“坏状态”



结论 调度器增强锁因果局部性和状态记忆；平台提供非对称 memory/I/O 路径。双稳态来自二者耦合，不能归因于单层。

干预一：DB node 决定档位，client node 几乎不决定

128 threads，mean TPS（每格 2 runs）



3.58×
DB node0 / DB node1 TPS

1.3% 以内
固定 DB 后交换 client 的差异

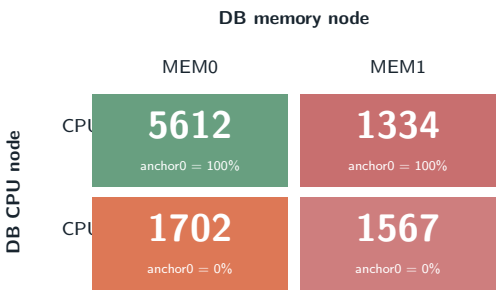
因果判断

改变 DB 放置直接切换档位；改变 client 放置不切换档位。H1 被否定，H2 获得干预支持。

数据来源：四格均使用 `scx_lockcoop -A -m10 -t2 -M`；warmup 10 s，measure 30 s；作废并发冲突样本未计入。

干预二：拆开 DB CPU 与 MEM，否定“anchor 单独充分”

MariaDB mean TPS；括号内为 anchor@node0



关键反证：CPU0 + MEM1

holder/anchor 仍 100% 位于 node0，但 TPS 从 5612 降至 1334（仅 23.8%）。所以 anchor0 不是充分条件。

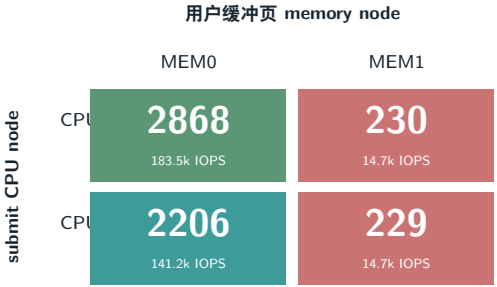
另一反事实：CPU1 + MEM0

I/O 缓冲页靠近设备侧，但 DB CPU 远端访问工作集，只有 1702 TPS。MEM0 也不是单独充分条件。

结论 唯一稳定高态是 CPU0+MEM0：compute、锁交接、工作内存与 node0 I/O 路径同时局部化。

子系统隔离：O_DIRECT 性能首先跟 memory node 走

4 GiB random read，MiB/s；128 threads，16 KiB，sync/direct



memory 主效应

CPU0 固定时，MEM0 → MEM1 使吞吐降低 12.49×；CPU1 固定时降低 9.63×。

CPU 次级效应

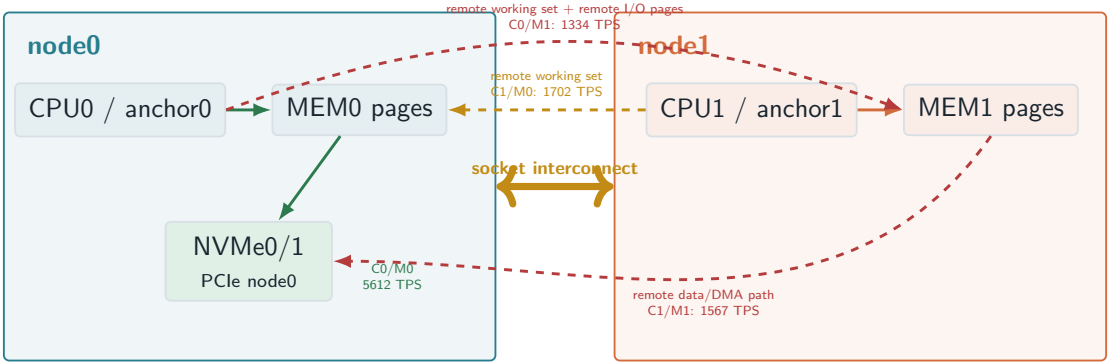
MEM0 固定时，CPU1 比 CPU0 低 23.1%；但仍远高于任意 MEM1 组合。

严谨表述

结果与“node0 NVMe 对 MEM1 页面发生远端 DMA/数据移动”一致，但尚未分解 IRQ、block-mq、IOMMU/SMMU 或固件贡献。

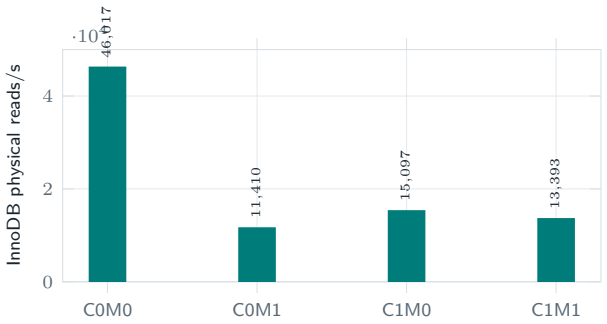
数据来源：CPU×MEM 四象限为补强诊断，终端 raw 未持久化；两轮本地复核和既有 2 GiB 实验均重复得到约 12×。

空间结构：同一块 NVMe，四种 CPU–MEM–device 路径并不等价



结论 “NVMe 挂在 node0”不是说 node1 不存在；它意味着任何落在 MEM1 的 I/O 页面都可能引入跨 socket 数据移动。

工作量没有变：每事务读页近似恒定，变化的是完成速率



Placement	reads/tx	TPS
C0/M0	8.614	5612
C0/M1	8.587	1334
C1/M0	8.789	1702
C1/M1	8.565	1567

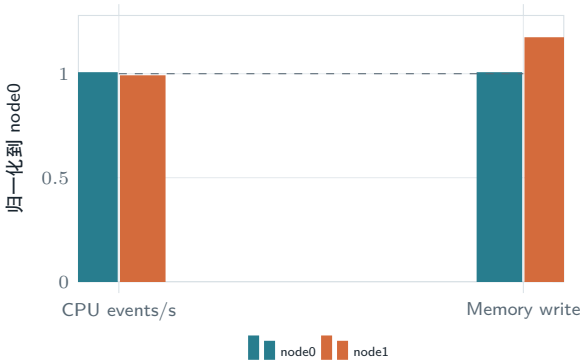
解释

每事务物理读仅相差约 2.6%，而 reads/s 相差约 4×。SQL 逻辑负载基本相同，I/O 完成能力决定 TPS 上限。

计数口径

status delta 覆盖 10 s warmup + 30 s measure，因此按 40 s 和两阶段总事务计算。

排除实验：第二颗 CPU 和本地内存本身并没有整体退化



Microbenchmark	node0	node1
CPU events/s	164664	162204
Memory MiB/s	490293	572671

排除结果

CPU 仅慢 1.5%；memory write 中 node1 反而更快。低态不是 socket1 整体算力故障。

异常范围

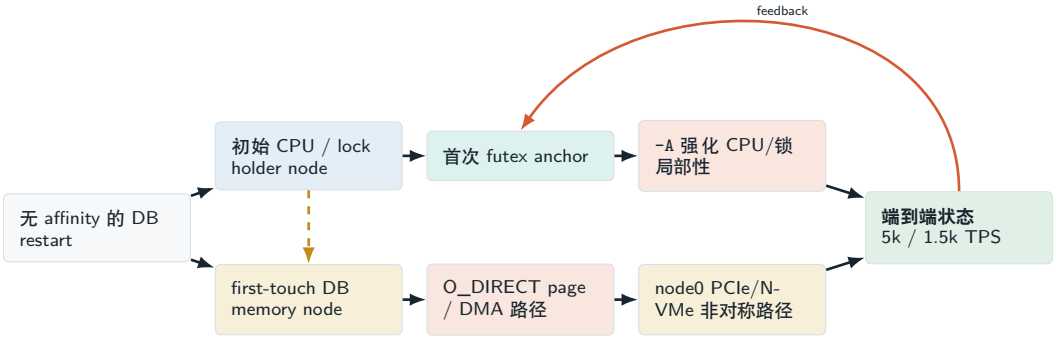
问题集中在 DB 工作集、锁交接、O_DIRECT 页面与 node0 PCIe/NVMe 的联合路径。

替代假设逐项检验

假设	反证/支持证据	判定
scheduler 偶发 attach 失败	高低态日志都成功 attach、持续产生 locality/forced 事件并正常 unregister	排除
SQL 工作量或错误不同	ignored errors/reconnects 均为 0；queries/tx 与 physical reads/tx 接近	排除
buffer pool 命中率是开关	pages_data 接近；每事务物理读接近；node0 冷启动仍进入高态	非主因
node1 CPU 降频/算力异常	两侧 3.0 GHz；CPU microbenchmark 仅差 1.5%	排除
sysbench client node 决定档位	DB/client 2×2 中，固定 DB 后交换 client 仅 1.3% 内变化	排除
anchor@node0 单独决定高态	CPU0/MEM1 中 anchor0=100%，但仅 1334 TPS	否定充分性
联合 CPU/lock/MEM/I/O 局部性	CPU0/MEM0 唯一达到 5.6k；四象限 fileio 与 DB 反事实方向一致	支持

结论 归因不是“相关图讲故事”，而是替代假设经过反事实实验后逐步收敛。

最终因果模型：已定位到联合路径，但不越界声称具体硬件子层



已证实 DB placement 可切换档位；仅联合局部态进入高态。

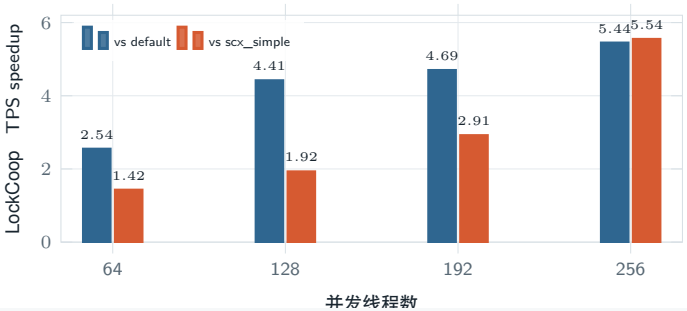
强支持 MEM node 主导当前 O_DIRECT 吞吐。

尚未细分 fabric/DMA/IOMMU/IRQ/block-mq/firmware。

4 RESEARCH NARRATIVE

结果解释、设计启示与后续验证

相对性能结论仍成立：高并发优势随压力扩大



5.44×
vs default @ 256T

5.54×
vs scx_simple @ 256T

解释限制 非绑定混合分布的 ratio of means 仅描述端到端结果，不是算法纯效应的无偏估计。

结论 跨硬件无需复现绝对 TPS；当前平台 256T 下相对 default/simple 达 5.44×/5.54×。

数据有效性审计：哪些结论可用，哪些必须作废或降级

数据集	状态	理由与使用方式
default / simple / cfl / lockboost / locknest / lockcoop 正式矩阵	有效	120 runs；有效 SCX 日志均确认 attach。用于端到端描述性比较。
scx_lcas 正式矩阵 20 runs	作废	verifier -E2BIG，20/20 未 attach；runner 超时后错误返回 0，数据实际是 disabled/default。主图已移除。
LockCoop 自然 20-run anchor 解析	有效	逐 run 识别本轮 DB TGID，排除系统 MariaDB；attach/unregister 完整。
DB/client 与 DB CPU/MEM 反事实	有效	每格 2 runs；NUMA pages、anchor、TPS/p95、InnoDB status 都持久化。
2 GiB 本地 O_DIRECT 复核	有效	两轮持久结果已写实验日志；11.6×。
4 GiB CPU×MEM fileio	诊断证据	方向由两轮本地复核支持，但四象限终端 raw 未持久保存；正式论文应重跑并落盘。

结论 LCAS 的独立 runner 缺陷不改变 LockCoop 双稳态归因，但正式七调度器比较必须补测 LCAS。

LockCoop 评价：收益与历史依赖风险

已经证明的能力

- 微基准中可缩短高争用锁交接路径；
- OLTP 高并发下明显优于 default / simple；
- 受控 CPU0/MEM0 状态可稳定达到 5.5k–5.9k TPS；
- 192/256T 仍保留显著吞吐和 p95 优势。

当前实现的体系结构风险

- 首次 holder/waker CPU 直接成为长期 anchor；
- `-m10` 统计更新，不验证真实 handoff；
- 热锁不断刷新 expiry，错误 anchor 难以失效；
- `-A` 忙 anchor 强制投递，形成强正反馈；
- 仅按 CPU topology，不理解 memory/device locality。

结论 LockCoop 的优势与风险来自同一个设计选择：用强状态记忆换取锁交接局部性。

调度器改进：从 CPU anchor 到联合局部性

P0 · 证据正确性 按目标 TGID/cgroup 过滤，不只按 `comm=mariadbd`；修复 futex waiter/owner 生命周期。

P1 · anchor 生命周期 使用经验证 handoff、滑动窗口与置信度；允许过期、衰减和动态 re-anchor。

P2 · 跨 node escape 检测持续忙、队列增长或吞吐下降后，允许跨 NUMA node 逃逸。

P3 · 拓扑目标函数 将 CPU、memory、device locality 与 IRQ/queue topology 纳入成本。

概念目标函数：$C = \alpha C_{handoff} + \beta C_{runqueue} + \gamma C_{cpu \leftrightarrow mem} + \delta C_{mem \leftrightarrow device}$

系数由在线观测更新；证据不足时退化到保守 CPU-topology policy，而非永久相信首次 anchor。

结论 研究机会不只是“修一个参数”，而是设计 lockcoop scheduling 与 memory/I/O topology 的协同控制。

正式评测协议应显式控制空间状态，而不是事后剔除样本

方案 A：受控公平比较

1. 所有 scheduler 使用相同 DB CPU/MEM placement；
2. client placement 保持一致；
3. 每 run 前核验 scheduler attach 与 DB TGID；
4. 记录 numastat、anchor ratio、IRQ/queue snapshot；
5. RUNS≥5，报告 raw、median、mean、CV 与置信区间。

方案 B：真正 unbound 鲁棒性研究

1. 保留无 affinity restart；
2. 把实际 CPU/MEM/anchor node 作为标签；
3. 分层报告高态、低态与迁移态概率；
4. 评价“进入有利 basin 的概率”和恢复时间；
5. 不以单个混合均值替代状态分布。

runner 必修复

`wait_scx_enabled()` 超时必须返回非零；启动后同时检查 PID、`sched_ext/state` 与 attach signature，失败立即终止 case。

下一阶段（一）：硬件 I/O 路径 A/B

待区分层级	A/B 干预	应采集证据与判定
PCIe root / socket fabric	单 NVMe、两盘分别测试；CPU/MEM 2×2；读取完整 PCIe tree	若两盘同方向且 MEM node 主导，支持共享 root/fabric 路径；记录带宽与延迟。
NVMe IRQ / block-mq queue	固定 IRQ affinity node0 vs node1；检查 queue cpu_list	采集 /proc/interrupts、每 vector 完成率、block issue/complete latency。
IOMMU/SMMU	当前参数 vs iommu.passthrough=1；iommu.strict=0/1	需一次性 grub next-boot A/B；比较 MEM1 IOPS 是否恢复，保留内核日志。
DMA page locality	pinned/registered buffers；fio libaio/io_uring 与 sysbench sync 对照	若不同 I/O engine 均随 MEM node 变化，增强 DMA/页面路径解释。

结论 所有需要改变 IOMMU/SMMU 启动参数的实验必须单独安排 reboot 窗口，并通过 BMC/控制台保留回退路径。

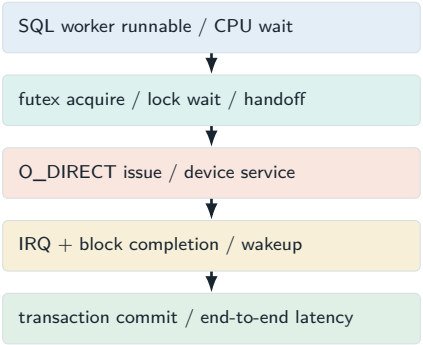
下一阶段（二）：建立 NUMA 与软件关键路径时间线

NUMA 工作集访问

在 C0/M1 与 C1/M0 反事实下采样 LLC、memory controller 与 socket-interconnect counters；修复当前 perf/traceevent 依赖，必要时使用平台 uncore/mesh 或厂商工具。

软件队列与锁

按实验 TGID 关联 `sched_switch`、futex、block issue/complete 与 IRQ；重建单事务 critical path。



结论 目标是把 3.6× 端到端差距分解为 CPU wait、lock wait、remote-memory 与 I/O completion 四类可量化区间。

贡献总结：一次端到端异常如何转化为体系结构结论

1 · 识别双稳态，而非“性能崩溃” 20-run anchor/TPS 相关性与 10 s 轨迹证明 2k–3k 是状态迁移。

▼

2 · 用干预确定模式开关 DB/client 2×2 证明档位跟 DB 放置走，client placement 影响很小。

▼

3 · 用反事实修正过强结论 CPU0/MEM1 否定 anchor 充分性；CPU×MEM 四象限指向联合局部性。

▼

4 · 定位到路径并保留边界 node0 NVMe + MEM1 的 O_DIRECT 约慢 10–12×；具体硬件子层仍待 A/B。

▼

5 · 形成系统设计启示 lock-aware scheduler 应纳入动态 anchor、memory/device topology 与实验自校验。

TAKEAWAY

LockCoop 的高并发收益是真实的；性能分叉来自它与非对称 NUMA/I/O 路径的强耦合。

高态需要 compute、锁交接、工作内存与 node0 NVMe 路径联合局部化。下一步不是删除低值，而是让调度器能够识别并逃离不利空间状态。

Q & A

5 RESEARCH NARRATIVE

附录与可复现性

附录 A：LockCoop 逐 run 原始值（64 / 128 threads）

threads	run	warmup TPS	measure TPS	p95 ms	anchor0	state
64	1	4458.71	5413.87	23.10	100.0%	high
64	2	2648.47	2436.23	44.98	97.7%	intermediate
64	3	1482.30	1509.16	71.83	0.0%	low
64	4	5745.77	5695.67	21.89	100.0%	high
64	5	1504.56	1517.64	69.29	0.0%	low
128	1	4930.42	4869.12	57.87	100.0%	high
128	2	1522.59	1545.96	137.35	0.0%	low
128	3	1534.10	1554.67	137.35	0.0%	low
128	4	5710.95	5758.90	44.98	100.0%	high
128	5	1510.55	1530.13	139.85	0.0%	low

结论 64/128T 的档位在 warmup 阶段已高度可预测；不是测量末尾的随机抖动。

附录 B：LockCoop 逐 run 原始值（192 / 256 threads）

threads	run	warmup TPS	measure TPS	p95 ms	anchor0	state
192	1	1237.67	1559.81	211.60	0.0%	low
192	2	5012.57	2828.83	196.89	64.7%	high → low
192	3	5386.42	2061.98	235.74	58.2%	migration
192	4	1537.77	1550.31	215.44	0.0%	low
192	5	1508.37	1507.13	223.34	13.6%	low
256	1	4381.23	3255.19	231.53	78.7%	high → low
256	2	1568.20	1574.93	282.25	0.0%	low
256	3	1568.59	1580.43	282.25	0.0%	low
256	4	1512.19	1549.77	292.60	0.9%	low
256	5	1402.34	1488.67	320.17	19.3%	low

结论 高并发中间值主要来自 run 内空间迁移；60 秒均值会丢失状态转移信息。

附录 C：正式矩阵 TPS 均值（RUNS=5）

scheduler	64T	128T	192T	256T	validity
default	1303.16	692.55	405.64	347.09	valid
scx_simple	2330.41	1585.44	652.83	341.24	valid
scx_lcas†	1256.78	644.21	452.51	325.94	invalid
scx_cfl	3021.67	1564.67	581.68	443.14	valid
scx_lockboost	3022.83	1831.11	667.16	482.33	valid
scx_locknest	2263.68	2147.89	1356.02	1442.67	valid
scx_lockcoop	**3314.51**	**3051.76**	**1901.61**	**1889.80**	valid

完整性

每个 scheduler/thread 目录均有 5 个 sysbench 输出；但“文件存在”不等于调度器已 attach。

† LCAS

20/20 为 verifier `-E2BIG` 后的 fallback/default；数字仅保留审计，不可解释为 LCAS 性能。

附录 D：正式矩阵 p95 均值，ms（RUNS=5）

scheduler	64T	128T	192T	256T	validity
default	121.99	499.23	1302.83	1928.11	valid
scx_simple	60.38	159.33	851.18	1938.12	valid
scx_lcas†	127.45	543.49	1202.30	2084.40	invalid
scx_cfl	53.91	160.47	839.39	1582.11	valid
scx_lockboost	53.96	151.58	817.57	1517.35	valid
scx_locknest	70.60	139.03	359.01	360.76	valid
scx_lockcoop	**46.22**	**103.48**	**216.60**	**281.76**	valid

聚合语义

表中 p95 是五个 run-level p95 的算术均值。对于双峰分布，应同时展示逐 run p95；它不是把所有事务合并后重新计算出的全局 95 分位数。

附录 E：DB/client 2×2 与 DB CPU/MEM 2×2 原始值

DB node × client node

DB	client	r1 TPS	r2 TPS	mean	p95
0	0	5679.26	5544.23	5611.74	49.25
0	1	5510.10	5855.45	5682.77	44.58
1	0	1583.53	1585.82	1584.67	134.90
1	1	1566.32	1568.04	1567.18	136.12

DB CPU node × DB memory node

CPU	MEM	r1 TPS	r2 TPS	mean	p95
0	0	5679.26	5544.23	5611.74	49.25
0	1	1334.92	1332.45	1333.69	181.58
1	0	1656.52	1747.74	1702.13	163.04
1	1	1566.32	1568.04	1567.18	136.13

共同协议

MariaDB 10.6.27；128 MiB BP；O_DIRECT；128 threads；warmup 10 s；measure 30 s；`scx_lockcoop -A -m10 -t2 -M`；每格 2 runs。

附录 F：-A -m 10 -t 2 -M 的准确语义

参数	源码变量/路径	本次实验含义
-A	force_busy_anchor=true	anchor CPU 忙时仍强制投递；本次高低态正反馈最直接的开关。
-m 10	min_handoff_seq=10	lock-nest 至少更新 10 次后才启用 handoff placement；当前计数不是经过验证的真实锁交接数。
-t 2	high_contention_threshold=2	owner-progress 的高竞争阈值设为 2 waiters；**不是** 2 秒过期。
默认 -e 2000	nest_expiry_ns=2000 ms	热锁持续刷新 last_update_ts，因此错误 anchor 很难自然过期。
-M	mariadb_only_evidence=true	仅接受 comm==mariadbd 的锁证据；并不区分实验 TGID 与系统 MariaDB。
默认同 node	require_same_node=true	prev-CPU escape 只允许在 anchor 所在 NUMA node 内；-N 才会关闭该限制。

附录 G：源码与脚本证据索引

文件	行号	证据
lock-sched/tools/sched_ext/scx_lockcoop .bpf.c	249–262	首次 holder CPU 初始化 anchor。
同上	328–368	热锁更新 last_update_ts，通常保持 anchor。
同上	654–697	-m gating、same-node escape、-A forced anchor。
同上	209–220	-M 只检查 comm==mariadbd。
lock-sched/tools/sched_ext/scx_lockcoop .c	42–63, 234–283	命令行参数 help 与实际赋值。
test3-SysbenchOLTP/scripts/run_full_sweep.sh	109–170	enabled wait 超时仍返回 0；导致 LCAS attach 失败未传播。
同上	454–490	scheduler attach 后 restart DB，再 warmup/measure。
test3-SysbenchOLTP/scripts/run_mariadb106_home_unbound_matrix.sh	24–38	无 NUMA wrapper 的 sysbench 和 DB restart；LockCausal args。

附录 H：原始结果目录与报告

正式非绑定 RUNS=5

```
test3-SysbenchOLTP/result/paper53_unbound_no_nest_runs5_20260801_224447_*
```

有效 DB/client 2×2

```
diag_lc_node0local_t128_r2_20260802_*
diag_lc_node1local_t128_r2_20260802_*
diag_lc_db0_client1_clean_t128_r2_20260802_*
diag_lc_db1_client0_t128_r2_20260802_*
```

有效 DB CPU/MEM 反事实

```
diag_lc_db_cpu0_mem1_client0_t128_r2_20260802_*
diag_lc_db_cpu1_mem0_client1_t128_r2_20260802_*
```

详细记录

```
LockCoop  /notes/mariadb_oltp_lockcoop  _diagnosis_20260731.md
LockCoop  /notes/paper53_unbound_no_nest_runs5_results_20260802.md
```

附录 I：有效性威胁与结论适用范围

威胁	当前影响	处理/后续
每个诊断 cell 仅 2 runs 4 GiB 四象限 raw 未落盘	足以确认 3.6× 大效应和反事实方向，不足以估计窄置信区间机制补强可用，审计性弱于其他结果	正式引用前交替顺序 RUNS≥5 使用同一脚本重跑并保存 stdout、kernel state、IRQ 快照
系统 MariaDB 同机存在	`-M` 可能收集其证据；本轮 anchor 分析已按实验 TGID 过滤	正式实验停止系统实例或改 cgroup/TGID filter
perf 工具动态库不匹配	无法获取 PMU/uncore 计数，硬件子层尚未细分	修复 libtraceevent/perf，补 interconnect、cache、IRQ 计数
单一 ARM 平台	结论证明当前平台耦合，不自动外推所有 NUMA 主机	在 x86 双路、设备对称平台和另一 ARM 平台复测
算术均值混合双态	描述性相对提升有效，但不是纯 scheduler treatment effect	受控 placement 比较或按状态分层建模

附录 J：术语与汇报时的精确措辞

术语/说法	精确定义
“系统有 node1 吗？”	有。node1 包含 CPU 128–255 和约 502 GiB 本地内存。
“NVMe 在 node0”	sysfs 报告两个控制器的最近 NUMA node 为 0；不是 node1 不存在，也不是 node1 不能访问盘。
“node1 I/O 慢”	更准确：当前 node0 NVMe 与 MEM1 页面组合的 O_DIRECT 路径极慢；CPU node 是次级因素。
“LockCoop 崩溃”	不准确。scheduler 正常运行并保持一个不利联合 NUMA 状态，导致端到端低态。
“anchor 导致 5k”	过强。anchor 是自然联合放置的代理和强化器；CPU0/MEM1 已否定其充分性。
“根因已经完全定位”	已定位到 scheduler feedback × NUMA memory/I/O 非对称路径；尚未定位具体 PCIe/IRQ/IOMMU/SMMU/firmware 子层。